

Integration Testing

The next testing cycle focuses on Integration Testing, whose goal is to verify the interaction between controllers and their corresponding model classes. The integration tests conducted mainly target the Controller package, as controllers represent the point where business logic and system components collaborate.

AdministratorDashboardController + Administrator → Cashier

(adminCashierTest)

To test the integration between the AdministratorDashboardController logic and the Administrator class when working with Cashier objects, three integration tests were executed.

The first test verifies the ability of the Administrator to add a Cashier to the system and later remove it. Because the project does not include a DAO or persistence layer, the test directly checks the modification of the in-memory list of Cashier objects. This test passed successfully.

The second test verifies the ability of the Administrator to revoke and grant working permission to a Cashier. Since the permission change is applied directly to the Cashier object stored in memory, the test passed successfully.

The third test verifies the Administrator's ability to modify a Cashier's identifier. This operation is expected to update the identifier consistently across the system. However, because the project does not implement a DAO or any persistence mechanism, the identifier update is not fully propagated or maintained, causing the test to fail. Therefore, this failure is due to the absence of a DAO layer rather than an error in the test itself.

AdministratorDashboardController + Administrator → Manager

(adminManagerTest)

To test the integration between the `AdministratorDashboardController` logic and the `Administrator` class when working with `Manager` objects, several integration tests were executed.

The first tests verify the ability of the `Administrator` to add a `Manager` to the system and later remove it. Since the project does not include a DAO or persistence layer, these tests directly validate changes made to the in-memory list of `Manager` objects. These tests passed successfully.

Another test verifies the `Administrator`'s ability to modify a `Manager`'s identifier. The test checks whether the updated identifier is correctly reflected in the stored `Manager` object. Because this operation is performed directly on the in-memory object, the test passed successfully.

The next tests verify the `Administrator`'s ability to revoke and grant working permission to a `Manager`. The revocation test failed because the permission state of the `Manager` is not properly updated or initialized within the in-memory structure. This behavior highlights a limitation of the current implementation, where permission changes are not fully handled without a DAO or persistence mechanism. The permission granting test passed once the permission state was explicitly updated.

Additional tests verify the `Administrator`'s ability to manage multiple `Manager` objects simultaneously, ensuring that adding or removing one `Manager` does not affect others. These tests passed successfully, confirming correct handling of collections.

Finally, a test verifies the `Administrator`'s login functionality and task description. Both authentication and task description retrieval behaved as expected, and the test passed successfully.

CashirDashboardController + Cashier → Bill → Item

(cashierBillItemTest)

To test the integration between the `CashierDashboardController` logic and the `Cashier`, `Bill`, and `Item` classes, five integration tests were executed.

The first test verifies the ability of a `Cashier` to create a `Bill` containing a single `Item` and correctly associate it with the `Cashier` who generated it. This test passed successfully.

The second test verifies that the total amount of the Bill is calculated correctly based on the Item's selling price and the requested quantity. The calculated total matched the expected value, and the test passed successfully.

The third test verifies that the Item stock quantity decreases after a Bill is created. The test expected the Item quantity to decrease from 10 to 7 after selling 3 units. However, the actual quantity was reduced to 9, causing the test to fail. This indicates that the stock reduction logic is not fully applied when creating a Bill, likely due to incomplete integration between the Cashier, Bill, and Item classes or missing stock-update handling in the absence of a DAO or persistence layer. Therefore, this failure is due to missing or partial business logic rather than an error in the test itself.

The fourth test verifies that the Cashier's total amount for the day is updated after creating a Bill. The daily total was updated correctly, and the test passed successfully.

The final test verifies that the Cashier's bill counter increases with each Bill creation. After creating multiple Bills, the counter reflected the correct number, and the test passed successfully.

CashierDashboardController + Cashier → Bill → Item (Stock Handling)

(cashierBillStockTest)

To test the integration between the CashierDashboardController logic and the Cashier, Bill, and Item classes with a focus on stock handling, an integration test was executed.

The test verifies that when a Cashier creates a Bill containing an Item, the Item's stock quantity is reduced accordingly. After selling two units of the Item, the test checks that the stock quantity decreases from its initial value and that a valid Bill object is created. Additionally, the test verifies that the Cashier's bill counter is incremented correctly.

All assertions were satisfied, and the test passed successfully, confirming that the stock update logic functions correctly in this specific scenario when creating a Bill.

ManagerController + Manager → Cashier

(cashierManagerTest)

To test the integration between the ManagerController logic and the Manager class when working with Cashier objects, several integration tests were executed.

The first test verifies the Manager's ability to add a Cashier to the system. The test confirms that the Cashier is correctly added to the Manager's internal list and that the Cashier identifier is stored accurately. This test passed successfully.

The second test verifies the removal of a Cashier by the Manager. After adding a Cashier, the Manager removes it using the Cashier's identifier, and the test confirms that the list of Cashiers becomes empty. This test passed successfully.

The next tests focus on the Manager's evaluation logic for Cashiers. One test verifies the ability to rate a Cashier who has not made any sales, ensuring that a valid evaluation string is returned. Another test checks the evaluation after the Cashier has completed sales by increasing the total amount won. In both cases, the returned rating was valid and non-null, confirming correct interaction between the Manager and Cashier classes. These tests passed successfully.

The final test verifies the Manager's ability to handle multiple Cashiers. After adding two Cashiers, one is removed while the other remains unaffected. The test confirms that the correct Cashier remains in the Manager's list, demonstrating proper list management and object handling. This test passed successfully.

Inventory + Item + Cashier

(inventoryItemCashierTest)

To test the integration between the Inventory, Item, and Cashier classes, several integration tests were executed to verify correct interaction during sales and reporting operations.

The first test verifies the initial state of the Inventory. It checks that the number of items stored, the total sold quantity, and the internal item list are initialized correctly when the Inventory is created. This test passed successfully.

The second test verifies the interaction between the Cashier and Inventory after a sale is performed. When the Cashier creates a bill and sells a quantity of an Item, the Item's stock is expected to decrease accordingly. However, this test failed because the Inventory does not actively track or update sold quantities when a sale is performed by the Cashier. Since the project does not include a DAO or a shared persistence mechanism, the Inventory and Cashier operate on loosely coupled data, which causes the Inventory state to not fully reflect changes performed by the Cashier. Therefore, the failure is due to missing integration logic between Inventory and Cashier rather than an error in the test itself.

The final test verifies the Inventory's reporting functionality over a given time period. After a sale is created by the Cashier, reports are requested for dates outside the sale period. Since no valid sales fall within the specified date ranges, the Inventory correctly returns a value of zero. This test passed successfully.

ManagerController + Manager → Item → Sector

(managerItemTest)

To test the integration between the Manager, Item, and Sector classes, a set of integration tests was conducted to verify that a Manager can correctly manage products inside a Sector.

The first test verifies the Manager's ability to add a new Item to a Sector. When the Manager adds a product, the Item is correctly inserted into the Sector's list of items, and the item name matches the expected value. This confirms correct interaction between the Manager and Sector classes. This test passed successfully.

The second test verifies the Manager's ability to update the quantity of an Item inside a Sector. After adding the Item, the Sector updates the corresponding quantity value correctly, confirming that stock management logic is properly integrated. This test passed successfully.

The final test verifies the Manager's ability to remove an Item from a Sector. However, this test failed with an `UnsupportedOperationException`. The failure occurs because the

Sector's item list was initialized using `List.of(...)`, which creates an immutable (unmodifiable) collection. As a result, when the `removeProductFromSector` method attempts to remove an Item, Java throws an exception since modifications are not allowed on immutable lists.

This failure is therefore caused by the use of an unmodifiable collection, not by incorrect integration logic or test implementation. The integration logic itself is correct, but the underlying data structure choice prevents item removal. Using a mutable collection such as `ArrayList` would resolve this issue.

Supplier + Item + Cashier

(supplierItemCashierTest)

To test the integration between the Supplier, Item, and Cashier classes, several integration tests were conducted to verify that supplier information is correctly associated with items and that item sales affect supplier-related data as expected.

The first test verifies that an Item is correctly linked to its Supplier. The test checks whether the supplier associated with the item matches the expected Supplier object and whether the supplier is marked as active for the store. This test failed because the Supplier reference is not consistently maintained or returned as the same object instance, even though the supplier data is logically correct. As a result, the object equality check fails, despite the relationship being conceptually valid.

The second test verifies that, after a sale performed by a Cashier, the Item quantity is reduced accordingly and that the Supplier's sold product count can be updated. This test failed because the Supplier's total sold quantity is not automatically updated by the bill creation process. The Cashier and Bill logic correctly updates the Item quantity, but no integration exists that propagates this change to the Supplier class. Therefore, the failure highlights a missing integration link rather than an incorrect test.

The final test verifies the ability to modify Supplier data, such as the supplier name and availability status. These updates are applied directly to the Supplier object and do not depend on other components. This test passed successfully.

